CS-2006 Operating Systems

Project List (Spring-2026)

1. Linux System Call Tracer (Mini strace)

Overview:
This project involves building a lightweight system call tracing tool similar to Linux strace. The tracer attaches to a running process or launches a new one, intercepting system calls using kernel debugging interfaces. Students learn how user programs interact with the OS kernel, observe system-level behaviour, and analyse performance and security implications of system calls.

Mandatory:

- Attach to a process using ptrace()
- Capture and log system call entry and exit
- Display system call names and return values
- Handle process creation and termination correctly
- Support tracing a newly spawned process

Additional – any 5

- Argument decoding for common system calls
- Timestamped system call logging
- Filtering specific system calls
- Per-process statistics (counts, time spent)
- Output redirection to log file
- Child process tracing (fork, exec)
- Coloured or formatted output

Final Product:
Command-line tool that prints a live system call trace with summaries and optional logs.

2. Cloud Task Orchestration Engine

Overview:

This project simulates a lightweight cloud task scheduler inspired by modern container orchestration systems. Multiple client threads submit compute tasks to a shared task pool. Worker threads fetch and execute tasks concurrently. Mutexes protect shared queues and task metadata, while semaphores limit the number of active workers. The system demonstrates thread pools, fairness, load balancing, and starvation avoidance. Students observe how OS-level scheduling decisions affect throughput and latency, closely resembling real cloud backend behavior.

Mandatory:

- Shared task queue with mutex
- Fixed-size worker thread pool
- Semaphore-controlled execution slots
- Correct task completion reporting

Additional – any 5

- Priority-based scheduling
- Task timeout handling
- Dynamic worker scaling
- Logging and monitoring thread
- Performance statistics

Final Product: GUI dashboard showing queued, running, and completed tasks.

3. Custom Command Line Shell

Overview:
This project builds a basic UNIX-like shell that accepts user commands, parses input, and executes programs using system calls. It mimics real shell behavior and introduces process creation, synchronization, and job control.

Mandatory:

- Read and parse user input
- Execute commands using fork() and exec()
- Parent process waits using wait() or waitpid()
- Handle invalid commands gracefully
- Support absolute and relative paths

Additional– any 5

- Built-in commands (cd, exit, pwd)
- Command history
- Background execution (&)
- Input/output redirection
- Pipes (|)
- Signal handling (Ctrl+C)
- Environment variable expansion

Final Product:
Interactive shell program with basic UNIX command functionality.

4. Autonomous Ride-Sharing Dispatch System

Overview:

This system models a ride-sharing backend where customer ride requests and drivers operate concurrently. Each request is a producer thread, and drivers are shared resources. Mutexes protect driver availability lists, while semaphores limit dispatch operations. The project highlights race conditions, matching logic, and synchronization between competing requests. It mirrors real-world dispatch systems used in ride-hailing platforms.

Mandatory:

- Ride request threads
- Shared driver pool
- Mutex-protected matching
- Deadlock-free dispatch logic

Additional:

- Ride cancellation
- Driver priority levels
- Surge pricing simulation
- Activity logging
- Performance metrics

Final Product:

Visual interface showing request-to-driver assignments.

5. Microkernel Message Passing System

Overview:
This project implements a user-space inter-process communication (IPC) framework inspired by microkernel architectures. Processes communicate exclusively through message passing rather than shared memory, highlighting isolation and modular OS design.

Mandatory

- Message send and receive primitives
- Process registration and addressing
- Blocking receive semantics
- Thread-safe message queues
- Error handling for invalid destinations

Additional – any 5

- Non-blocking message operations
- Message priorities
- Timeouts for receive
- Broadcast or multicast messages
- Message logging
- Performance benchmarking
- Capability-based access control

Final Product:
IPC library with demo applications showing client-server communication.

6. Multi-Threaded Producer–Consumer Problem

Overview:
This project simulates multiple producer and consumer threads sharing a bounded buffer. It

demonstrates synchronization, race condition prevention, and coordination using mutexes and semaphores.

Mandatory

- Fixed-size shared buffer
- Multiple producer threads
- Multiple consumer threads
- Mutual exclusion using mutexes
- Proper synchronization to avoid over/underflow

Additional – any 5

- Variable production/consumption rates
- Logging thread activity
- Dynamic buffer size
- Graceful shutdown
- Thread statistics
- Fair scheduling
- Visualization of buffer state

Final Product:
Console or GUI simulation showing producer-consumer interaction in real time.

7. Parallel Log Processing & Analytics Engine

Overview:

Large log files are divided into segments and processed by multiple threads concurrently. Each thread parses logs and updates shared statistics. Mutexes ensure safe updates to global counters, while semaphores control disk access. This project teaches parallel file processing, synchronization overhead, and real-time analytics, similar to security monitoring systems.

Mandatory:

- Log segmentation
- Concurrent parsing threads
- Mutex-protected statistics
- Accurate result aggregation

Additional:

- Pattern detection
- Severity classification
- Real-time graphs
- Report export
- Speed comparison

Final Product: Analytics panel with charts and summaries.

8. Smart Energy Grid Load Balancer

Overview: This project simulates an energy grid where generators supply power and regions consume it concurrently. Semaphores control total capacity, and mutexes protect shared grid state. The system demonstrates bounded resources, fairness, and starvation prevention, reflecting smart grid management.

Mandatory:

- Producer–consumer model
- Semaphore-based capacity control
- Mutex-protected grid state
- Load balancing logic

Additional:

- Peak-hour simulation
- Priority regions
- Fault simulation
- Logging
- Graph visualization

Final Product: Live grid visualization showing load distribution.

9. User-Level Thread Library

Overview:
Students implement a cooperative threading library entirely in user space. Context switching is handled manually without kernel support, offering deep insight into thread internals and scheduling mechanics.

Mandatory

- Thread creation API
- Context switching using getcontext() / setcontext()
- Yielding between threads
- Thread termination
- Thread join functionality

Additional – any 5

- Preemptive scheduling (timer-based)
- Priority scheduling
- Round-robin scheduler
- Thread-local storage
- Stack overflow detection
- Debugging tools
- Performance metrics

Final Product:
Reusable user-level thread library with test applications.

10. Parallel Online Examination System

Overview:

Multiple students attempt exams simultaneously while evaluator threads grade responses in parallel. Mutexes protect result storage, and semaphores limit grading capacity. This project reflects real LMS backends and highlights synchronization under heavy read/write loads.

Mandatory:

- Student submission threads
- Evaluator threads
- Mutex-protected results
- Semaphore-controlled grading

Additional:

- Time limits
- Auto-save
- Cheating detection
- Logs
- Analytics

Final Product: Exam dashboard with live status.

11. OS-Level Chat Server Using Thread Pools

Overview:

A concurrent chat server where clients are handled using a thread pool. Shared chat rooms require mutex-protected message buffers, while semaphores manage connection limits. This project demonstrates server-side concurrency and synchronization.

Mandatory:

- Thread pool implementation
- Client request handling
- Mutex-protected message buffers
- Safe broadcast mechanism

Additional:

- Private messaging
- Authentication
- Logging
- Rate limiting
- Emoji support

Final Product: Chat UI with rooms and users.

12. Smart Parking Allocation System

Overview:

This project simulates a smart parking management system where multiple vehicles arrive concurrently and request parking slots. Each vehicle is represented by a thread, while parking slots are shared limited resources. Mutexes protect the parking slot table, and semaphores control the total number of available slots. The system demonstrates classic resource allocation, race condition handling, and fairness. Students learn how improper synchronization may lead to double-allocation or starvation. This project reflects real smart-city parking systems.

Mandatory :

- Vehicle threads requesting slots
- Semaphore for available parking slots
- Mutex-protected slot allocation table
- Safe entry and exit handling

Additional Features:

- Priority parking (VIP/emergency)
- Waiting queue visualization
- Parking timeout
- Slot release logging
- Utilization statistics

GUI + Viva : GUI shows parking layout and occupied/free slots. Viva focuses on semaphore use.

Final Product: Live parking dashboard.

13. Distributed File Backup Manager

Overview:

This project models a distributed backup system where multiple client threads send files to be backed up on a shared storage server. Backup worker threads process file chunks concurrently. Mutexes protect metadata and backup indexes, while semaphores limit concurrent disk writes. The project highlights synchronization in I/O-heavy systems and mirrors enterprise backup solutions.

Mandatory:

- Client backup request threads
- Worker threads for backup
- Mutex-protected metadata
- Semaphore-limited disk access

Additional Features:

- Incremental backups
- Failure recovery
- Compression simulation
- Backup logs
- Progress tracking

GUI + Viva (30%): GUI displays backup status per client. Viva discusses I/O synchronization.

Final Product: Backup management console.

14. Parallel Image Processing Pipeline

Overview: This project implements a multi-stage image processing pipeline where images pass through stages such as loading, filtering, enhancement, and saving. Each stage is handled by a separate thread pool, and images move through shared buffers. Mutexes protect buffers, while semaphores manage pipeline capacity. The project demonstrates pipeline parallelism and synchronization between stages.

Mandatory :

- Multi-stage pipeline design
- Mutex-protected buffers
- Semaphore-based stage control
- Correct image flow order

Additional Features:

- Multiple filter options
- Batch processing
- Performance comparison
- Error handling
- Progress visualization

GUI + Viva : GUI shows pipeline stages and image status. Viva focuses on pipeline synchronization.

Final Product: Image processing workflow viewer.

15. Producer–Consumer Using Semaphores

Overview:
This project focuses on solving the classic producer–consumer problem strictly using synchronization primitives. It emphasizes correctness, deadlock prevention, and critical section management.

Mandatory

- Semaphore-based synchronization
- Mutual exclusion for shared buffer
- Correct handling of empty/full conditions
- Multiple producers and consumers
- Deadlock-free execution

Additional – any 5

- Monitor-based implementation
- Starvation prevention
- Fair semaphore usage
- Logging execution order
- Performance comparison with mutexes

- Visualization
- Error injection testing

Final Product:
Correct and well-documented concurrency simulation with trace output.

16. Banker's Algorithm for Deadlock Avoidance

Overview:
This project simulates dynamic resource allocation using Banker's Algorithm to avoid deadlocks. It models processes, resource requests, and system safety checks.

Mandatory

- Representation of resource types
- Allocation, maximum, and need matrices
- Safety algorithm implementation
- Request handling logic
- Deadlock avoidance verification

Additional – any 5

- Graphical visualization
- Randomized resource requests
- Interactive input
- Rollback simulation
- Logging safe/unsafe states
- Performance analysis
- Multi-threaded simulation

Final Product:
Simulator that shows safe sequences and system states.

17. AI Model Training Job Scheduler

Overview: This project simulates an AI training server where multiple training jobs are submitted concurrently. Each job consumes limited GPU/CPU resources. Mutexes protect job queues and resource tables, while semaphores limit concurrent training sessions. The project introduces advanced scheduling and resource contention scenarios common in ML platforms.

Mandatory :

- Job submission threads
- Shared resource table
- Mutex-protected scheduling
- Semaphore-controlled execution

Additional Features:

- Job priority levels
- Preemption simulation
- Runtime estimation
- Logging
- Resource utilization charts

GUI + Viva (30%): GUI shows job queue and resource usage. Viva discusses fairness and starvation.

Final Product: Training scheduler dashboard.

18. Railway Signal Coordination System

Overview: This project simulates railway signaling where multiple trains attempt to use shared track segments. Each train is a thread, and track segments are shared resources. Mutexes ensure exclusive track access, and semaphores coordinate signal permissions. The project demonstrates deadlock prevention and safe sequencing.

Mandatory:

- Train threads
- Track segment locking
- Semaphore-based signaling
- Deadlock-free routing

Additional Features:

- Priority trains
- Delay simulation
- Logging
- Route visualization
- Throughput analysis

GUI + Viva (30%): GUI shows tracks and train movement. Viva focuses on deadlock avoidance.

Final Product: Railway signal simulator.

19. Real-Time Auction Bidding Platform

Overview:

This project models a real-time online auction system where multiple bidders place bids concurrently on shared items. Each bid is a thread, and items are shared resources. Mutexes protect highest-bid data, and semaphores limit active bidding sessions. The project demonstrates race conditions and atomic updates.

Mandatory :

- Bidder threads
- Mutex-protected item data
- Semaphore-controlled auctions
- Atomic bid updates

Additional Features:

- Bid timeout
- Auto-bidding
- Auction history
- Notifications
- Statistics

GUI + Viva: GUI shows live bids and winners. Viva focuses on atomicity and critical sections.

Final Product: Live auction dashboard.

20. Smart Warehouse Robot Coordinator

Overview: This project simulates a smart warehouse where multiple autonomous robots move concurrently to pick and place items. Each robot is implemented as a thread, while storage zones and paths are shared resources. Mutexes protect shared maps and item locations, and semaphores limit the number of robots allowed in critical zones to avoid collisions. The project demonstrates synchronization in spatially shared resources, deadlock avoidance, and fairness. Students learn how OS concepts apply to robotics coordination and industrial automation systems such as automated fulfillment centers.

Mandatory:

- Robot threads with concurrent execution
- Shared warehouse map
- Mutex-protected item access
- Semaphore-controlled critical zones

Additional Features:

- Robot priority handling
- Collision detection simulation
- Task queue optimization
- Activity logging
- Throughput analysis

GUI + Viva : GUI visualizes robot movement and zones. Viva focuses on deadlock prevention.

Final Product: Animated warehouse coordination system.

21. Parallel Web Search Crawler

Overview:

This project models a simplified web search crawler where multiple crawler threads fetch and index web pages concurrently. Shared data structures store visited URLs and keyword indexes. Mutexes ensure consistency of indexes, while semaphores limit the number of concurrent fetch operations. The project demonstrates synchronization in networked applications and parallel search systems.

Mandatory:

- Crawler threads
- Shared URL repository
- Mutex-protected index updates
- Semaphore-limited crawling

Additional Features:

- Duplicate URL detection
- Keyword ranking
- Depth-limited crawling
- Crawl statistics
- Error handling

GUI + Viva: GUI shows crawling progress and indexed pages. Viva discusses race conditions.

Final Product: Search crawler monitoring dashboard.

22. Online Stock Trading Simulator

Overview: This project simulates an online stock trading platform where multiple traders place buy and sell orders concurrently. Each order is a thread, while stock order books are shared resources. Mutexes protect order books, and semaphores limit active trading sessions. The project demonstrates atomic transactions, consistency, and synchronization under high contention, similar to real trading engines.

Mandatory:

- Trader/order threads
- Shared order book
- Mutex-protected trade execution
- Semaphore-controlled trading

Additional Features:

- Market vs limit orders
- Price fluctuation simulation
- Trade history
- Profit/loss analytics
- Logging

GUI + Viva: GUI shows live prices and trades. Viva focuses on atomicity.

Final Product: Stock trading dashboard.

23. Smart Classroom Resource Manager

Overview: This project manages shared classroom resources such as projectors, labs, and multimedia equipment. Multiple class sessions request resources concurrently. Mutexes protect resource tables, and semaphores limit availability. The project demonstrates fair allocation and starvation avoidance.

Mandatory:

- Session request threads
- Shared resource pool

- Mutex-protected allocation
- Semaphore-based limits

Additional Features:

- Priority scheduling
- Reservation system
- Usage logs
- Conflict alerts
- Statistics

GUI + Viva: GUI shows resource allocation timeline. Viva focuses on fairness.

Final Product: Classroom resource scheduling panel.

24. Multi-Process Video Encoding System

Overview: This project simulates a video encoding pipeline where multiple processes handle encoding tasks concurrently. Each process uses internal threads for frame processing. Mutexes protect shared output buffers, and semaphores coordinate stage completion. The project introduces students to multi-process plus multi-thread synchronization.

Mandatory:

- Multi-process architecture
- Thread-based frame encoding
- Mutex-protected output buffers
- Semaphore-based stage sync

Additional Features:

- Different encoding profiles
- Load balancing
- Performance metrics
- Error recovery
- Logging

GUI + Viva: GUI shows encoding progress. Viva covers process vs thread differences.

Final Product: Video encoding monitor.

25. OS-Level Firewall Packet Processor

Overview:

This project simulates an OS-level firewall where incoming packets are processed concurrently. Each packet is handled by a thread, while shared rule tables determine allow/deny decisions. Mutexes protect rule updates, and semaphores limit packet processing rates. The project demonstrates concurrency in low-level system services.

Mandatory:

- Packet processing threads
- Shared rule table

- Mutex-protected rule access
- Semaphore-based rate control

Additional Features:

- Dynamic rule updates
- Packet logging
- Priority packets
- Statistics dashboard
- Attack simulation

GUI + Viva GUI shows packet flow and decisions. Viva focuses on critical sections.

Final Product: Firewall activity monitoring tool.

26. Multi-threaded File Compression System

Overview: This project simulates a parallel file compression tool similar to gzip or WinRAR. Multiple threads read chunks of a large file and compress them concurrently using a chosen compression algorithm. Mutexes ensure exclusive access to shared file metadata, and semaphores control the number of threads writing to the output file. The system demonstrates concurrent I/O, thread synchronization, and buffer management, providing insights into how OS-level threading can accelerate computationally heavy tasks in file systems.

Mandatory:

- Threaded reading of file chunks
- Mutex-protected output writing
- Semaphore-controlled compression threads
- Correct reconstruction of original file on decompression

Additional:

- Priority-based chunk processing
- Dynamic buffer resizing
- Compression ratio reporting
- Pause/resume feature
- GUI showing thread progress and compressed size

Final Product: A GUI with file selection, compression progress bars, decompression validation, and thread activity display.

27. Restaurant Order Management Simulator

Overview:

This project models a multi-threaded restaurant kitchen system where waiter threads submit customer orders to a shared queue, and chef threads prepare orders concurrently. Mutexes prevent race conditions on shared order queues, while semaphores limit the number of active chefs in the kitchen. This demonstrates producer-consumer synchronization, real-time scheduling, and resource

allocation under load. It simulates real-life order management and kitchen optimization in the restaurant industry.

Mandatory:

- Shared order queue with mutex
- Multiple waiter and chef threads
- Semaphore-controlled kitchen stations
- Order completion notification system

Additional:

- Priority-based order processing (VIP customers)
- Dynamic chef allocation
- Order cancellation and re-prioritization
- Real-time kitchen load monitoring

GUI with live order status and chef activity

Final Product: A GUI dashboard showing incoming orders, chef assignment, and completion status in real-time.

28. **Concurrent Banking Transaction System**

Overview: This project simulates a multi-threaded banking system where multiple clients perform transactions on shared accounts. Threads perform deposits, withdrawals, and transfers, using mutexes to prevent race conditions and semaphores to limit concurrent access per account. Students observe issues like deadlocks, starvation, and consistency, which are critical in real-world financial systems.

Mandatory:

- Thread-safe account operations with mutex
- Semaphore control for maximum concurrent transactions
- Transaction logging
- Balance consistency verification

Additional – any 5:

- Priority-based VIP transactions
- Undo/rollback failed transactions
- Dynamic account creation
- Fraud detection simulation
- GUI showing accounts, transactions, and logs

Final Product: GUI displaying accounts, transaction threads in action, success/failure notifications, and transaction history charts.

29. **Multi-threaded Traffic Signal Controller**

Overview:

This project simulates a traffic intersection with multiple signals and lanes. Each lane thread controls vehicle flow while semaphores manage critical intersections to prevent collisions. Mutexes ensure safe updates to shared traffic statistics. Students explore thread coordination, synchronization, and timing, which reflects real-time embedded system behavior in traffic management.

Mandatory:

- Lane threads controlling signal lights
- Semaphore-protected intersection crossing
- Shared traffic statistics updated safely with mutex
- Correct vehicle throughput computation

Additional – any 5:

- Adaptive traffic signal timing
- Emergency vehicle priority
- Pedestrian crossing thread
- Dynamic vehicle generation
- GUI visualization with moving vehicles and signals

Final Product: GUI showing intersection layout, moving vehicles, signal states, and live throughput metrics.

30. **Multi-threaded Online Quiz System**

Overview:

This project models an online exam system where multiple students (threads) take quizzes simultaneously. Shared resources like question banks and scoreboards are protected by mutexes, while semaphores limit concurrent database writes. The system demonstrates concurrency control, thread-safe data handling, and synchronization challenges in online education platforms.

Mandatory

- Thread-safe question access
- Mutex-protected score updates
- Semaphore-controlled submission queue
- Accurate score computation per student

Additional – any 5:

- Randomized question selection
- Timed quizzes with countdown
- Analytics dashboard for scores
- Multi-admin thread management

- GUI showing live quiz progress and leaderboard

Final Product: GUI displaying questions, countdown timers, student scores, and live leaderboard updates.

31. Readers–Writers Toolkit

Overview:
This project implements classic Readers–Writers synchronization problems, focusing on fairness, starvation prevention, and concurrent access control.

Mandatory

- Reader and writer thread implementation
- Mutual exclusion for writers
- Concurrent reader access
- Correct synchronization primitives
- Deadlock-free execution

Additional– any 5

- Reader-priority variant
- Writer-priority variant
- Fair solution
- Execution timeline visualization
- Logging and tracing
- Configurable thread counts
- Performance comparison

Final Product:
Visualization tool or simulator demonstrating access patterns.

32. Thread Scheduler for Educational OS

Overview:
Students implement a CPU scheduling algorithm inside a small educational operating system kernel. This project bridges theory and real kernel-level scheduling.

Mandatory

- Implement round-robin or priority scheduler
- Maintain runnable process queue
- Context switching logic
- Timer interrupt handling
- Process state transitions

Additional – any 5

- Multi-level feedback queue
- Priority aging
- Visualization of scheduling
- Load balancing
- Configurable time slices
- Starvation prevention

Final Product:
Modified kernel demonstrating custom scheduling behaviour.

33. Deadlock Detection Tool

Overview:
This project builds a runtime deadlock detection system using wait-for graphs. The tool identifies circular dependencies among processes and reports deadlocks.

Mandatory

- Resource allocation tracking
- Wait-for graph construction
- Cycle detection algorithm
- Deadlock reporting
- Dynamic updates on request/release

Additional – any 5

- Graph visualization
- Recovery strategies
- Periodic detection
- Logging and alerts
- Simulation mode
- Performance optimization

Final Product:
Deadlock detection simulator with clear reports and visual graphs.

Note:

1. **For system call projects, kernel configuration should be done with student ID**
2. **Above all Projects have following Parts with Marks Distribution**
    a. Mandatory = 40%
    b. Additional = 40%
    c. Interface(GUI/CLI) = 20%
    d. Viva 20%

Requirements:

Group Instructions:

- Maximum 3 Group members

- Cross-section of the same teacher of the class is allowed ONLY
- The same project is not allowed within a section and approval will be granted on an FCFS basis.
- Please note that there is only one project in the course. project marks will be put in theory.
- Only C or C ++ language is preferred. No python-based projects in OS.

Deliverables and Due Dates:

- Project Proposal (project title, introduction, methodology) submission on 7th week
- Final Demonstration with the following requirements:
- Final Working Project Demo
- Project Viva
- Final Report soft copy (objectives, project details, results (comparison via graphs), conclusion.
- Code repository on GitHub is a must. Provide the link
- Note: Final project demonstration is tentatively scheduled from 15th week